Detecting Money Laundering with Transaction and Account Graph Features

Bokang GAO Zi Xuan SZETO Minghua XIONG
Group: zxszeto's group (zxszeto_group)

1 Introduction

Anti-money-laundering (AML) detection is not an ordinary balanced classification problem. A monitoring system must search through a very large legitimate transaction stream for a very small number of suspicious events, while every alert consumes investigation capacity. In this setting, a model can appear accurate by predicting almost everything as legitimate and still be useless. The practical objective is therefore not to automate a final legal judgment; it is to rank transactions and accounts so that scarce human review is directed toward more informative cases.

This project studies that problem using the IBM synthetic AML transaction dataset. The full dataset contains 5,078,345 transactions, 5,177 laundering labels, and a positive rate of 0.1019%. A transaction row records an event, but laundering behavior may be expressed across a sequence or network of accounts. Our central question is therefore: *do account graph features add useful context beyond transaction-level variables, and does any observed gain survive more realistic evaluation protocols?*

The study makes three contributions. First, it connects the course methods—preprocessing, t-SNE, clustering, supervised learning, formal cross-validation, and feature selection—in one reproducible rare-event pipeline. Second, it evaluates graph-aware behavior features through explicit ablations rather than assuming that network context must help. Third, it treats feature availability as part of the research question. A retrospective random graph, a frozen training snapshot, and a strict expanding history are deliberately separated because they represent different information sets. This distinction is decisive: a frozen snapshot fails under chronology, whereas transaction-by-transaction history restores a positive graph contribution without using future edges.

Table 1: Failure-driven experiment path.

Attempt	Observed failure	Next decision	Result
All-legitimate	High accuracy, zero recall	Use rare-event metrics	F1 and PR-AUC emphasized
LR / Linear SVM	Weak PR-AUC	Try nonlinear models	Tree/RF/HGB
Decision Tree	Many false positives	Use ensembles	More stable ranking
Base RF/HGB	No account context	Add graph features	Random-split PR-AUC improves
Natural prior	Precision collapses	Alert-budget sweep	Operational trade-off exposed
Frozen time snapshot	Graph lift reverses	Diagnose representation	History must update online
Strict expanding history	Prior edges only	Retune and retest	Time-test PR-AUC improves

The sequence in Table 1 reflects how the analysis actually evolved. The first baseline exposed the accuracy illusion. Linear models then provided interpretable reference points but left substantial nonlinear structure unexplained. A single tree recovered more positives at the cost of many false alerts, motivating ensembles. Graph features produced the strongest sampled-test result, but the natural-prior test exposed an unmanageable alert queue. The first time audit then failed because one training-period snapshot could not update for later transactions. We retained that negative result, rebuilt account and edge statistics before each transaction using only earlier rows, and retested. This failure–diagnosis–repair sequence is the central open-ended result.

2 Dataset and Experimental Protocol

The official dataset source is Kaggle/IBM. The scripts use a Hugging Face mirror only as a programmatic fallback. The data are synthetic, which makes complete labels and large-scale experimentation possible without exposing private banking records. At the same time, synthetic generation can simplify customer behavior, institutional controls, and adversarial adaptation. We therefore treat the dataset as a controlled benchmark for comparing methods, not as direct evidence of performance at a real bank.

Full-data statistics are computed in chunks so the reported base rate reflects all 5,078,345 rows. t-SNE and density-based clustering use samples for computational feasibility. Supervised model development uses a stratified modeling sample containing all laundering transactions and sampled legitimate transactions. This enrichment makes model comparison feasible and ensures that minority cases are represented, but it also changes the class prior. Consequently, sampled-test precision cannot be read as the precision of a production alert system.

Three evaluation families answer different questions. (1) The *random/transductive sampled evaluation* compares models and ablations, including training-only three-fold grid search. (2) The *natural class-prior stress test* restores the full-data base rate to expose alert pressure. (3) The *temporal evaluation* first records the failure of a frozen training snapshot and then tests strict expanding-history features computed before each transaction. These are separate protocols, not interchangeable views of one test set.

Table 2: Interpretation boundary for the three evaluation protocols.

Protocol	Primary question	What it supports	What it does not support
Random/transductive sample	Which model ranks better?	Controlled model/feature comparison	Deployment performance
Natural-prior stress	What happens when rarity returns?	Base-rate and alert-pressure diagnosis	Independent temporal validation
Frozen-snapshot audit	Why does temporal graph performance fail?	Representation mismatch diagnosis	Prospective graph value
Strict expanding history	Do prior-only updates help later rows?	Chronological feature ablation	Natural-prior simulation

Model thresholds are selected using validation data inside the training process and then held fixed for test evaluation. We emphasize precision, recall, F1, and especially PR-AUC because the positive class is rare. ROC-AUC remains useful as a ranking metric and is reported for completeness, but it can look strong even when the absolute number of false positives is operationally large. Confusion matrices and alerts per 10,000 transactions translate statistical performance into review workload.

3 Mandatory Tasks

3.1 Preprocessing and EDA

We check missing values and duplicates, parse timestamps, one-hot encode currency and payment format, and standardize numeric features. Engineered transaction-level features include amount, log amount, hour, weekday, same-bank flag, and same-currency flag. Log amount reduces the influence of a long monetary tail, while hour and weekday preserve temporal regularity that a raw timestamp cannot express. Same-bank and same-currency indicators provide compact relational context without requiring a full graph model.

The preprocessing design follows the training boundary: transformations are learned from training data and applied to held-out data. Account and bank identifiers are not treated as arbitrary continuous numbers. The goal is to retain behavior while avoiding meaningless numeric distance between IDs. Exploratory analysis is used to formulate hypotheses, not to select a result after seeing test labels. The most important EDA observation is the base rate itself: at 0.1019%, accuracy is structurally incapable of summarizing minority-class usefulness.

3.2 t-SNE Visualization

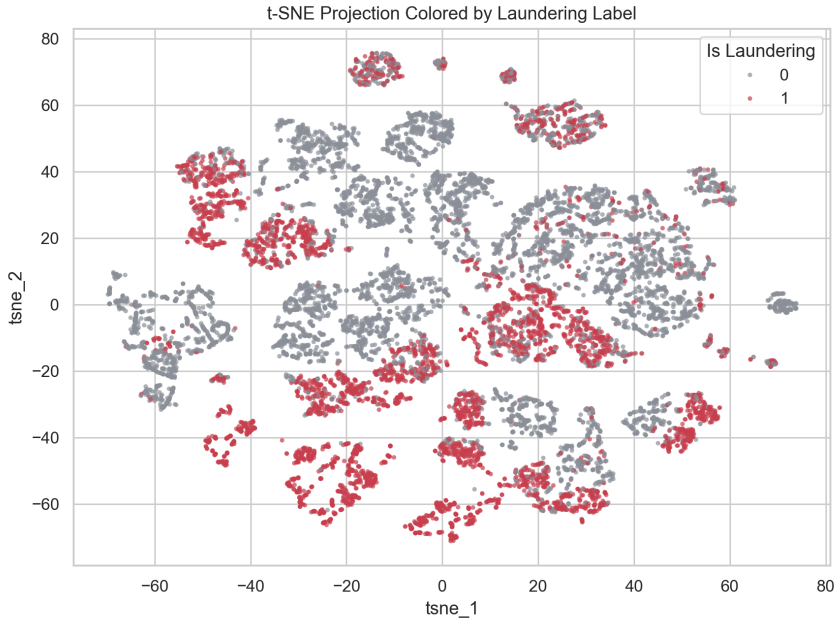
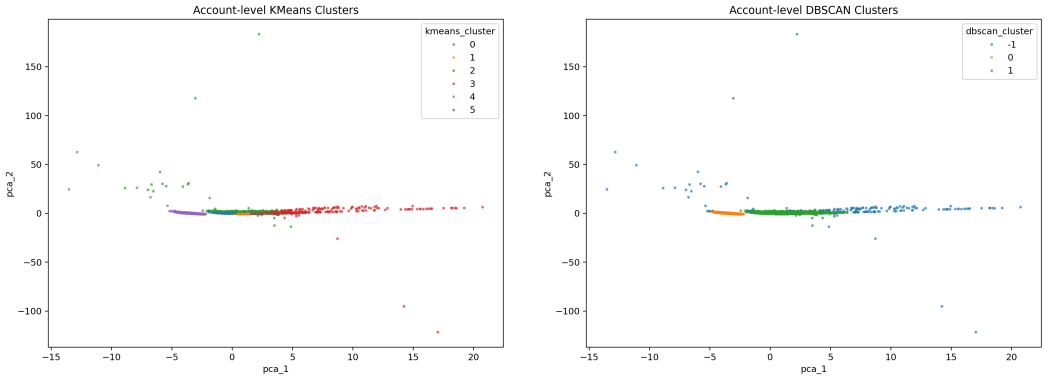


Figure 1: t-SNE projection on a stratified transaction sample. Laundering samples form local neighborhoods but still overlap with legitimate transactions, so visualization does not replace supervised learning.

The t-SNE map changes our initial expectation. Laundering points are neither uniformly random nor cleanly separated. They form several local concentrations but overlap substantially with legitimate transactions. This suggests that suspiciousness is conditional on combinations of features and that a single global linear boundary is unlikely to be sufficient. It also warns against using a visually compelling embedding as proof of classification quality: t-SNE preserves local neighborhoods, distorts global distance, and is produced on an enriched sample.

3.3 KMeans and DBSCAN Clustering

KMeans and DBSCAN are applied to account-level behavior features. Purity is inflated by class imbalance, so it must be interpreted together with NMI and ARI.



(a) Account-level KMeans clusters (b) Account-level DBSCAN clusters

Figure 2: Cluster-label visualizations using account-level behavior features. DBSCAN isolates a small high-risk noise/outlier group (label -1) in the modeling account sample, but both methods have very low NMI/ARI and therefore cannot replace supervised labels. Purity is also inflated by class imbalance.

Table 3: Account-level clustering metrics.

Method	Sil.	NMI	ARI	Purity	Max risk
Account KMeans	0.380	0.021	0.017	0.759	0.424
Account DBSCAN	0.478	0.001	-0.008	0.763	0.848

The clustering results reinforce this mixed picture. DBSCAN has a higher silhouette score than KMeans, indicating more coherent geometry among its non-noise regions, but both NMI and ARI are close to zero. Thus, cluster structure and laundering labels describe different partitions of the data. The maximum DBSCAN risk value of 0.848 belongs specifically to label -1, the small noise/outlier group in the modeling account sample. It is not 84.8% accuracy, overall purity, or natural-prior precision. Its value is investigative: the method isolates a compact set worth profiling, but it does not produce a reliable laundering classifier. This failure is useful because it motivates supervised models while preserving clustering as an exploratory triage tool.

3.4 Supervised Model Training and Validation

We compare Dummy All-Legitimate, Logistic Regression, Linear SVM, Decision Tree, Random Forest, and HistGradientBoosting. This sequence is intentionally progressive. The dummy model quantifies the accuracy trap; Logistic Regression and Linear SVM test whether a linear score is sufficient; the Decision Tree tests nonlinear interactions but is vulnerable to unstable partitions; Random Forest and HistGradientBoosting test whether ensembles can improve ranking and control variance. Thresholds are selected on a validation split inside the training set rather than tuned on the final test labels.

Table 4: Sampled-test model comparison.

Model	F1	ROC-AUC	PR-AUC
HistGradientBoosting + Graph Features	0.628	0.976	0.721
Random Forest + Graph Features	0.591	0.972	0.681
HistGradientBoosting Base Features	0.540	0.930	0.556
Random Forest Base Features	0.529	0.927	0.550
Decision Tree	0.469	0.906	0.405
Linear SVM	0.459	0.915	0.327
Logistic Regression	0.459	0.915	0.338
Dummy All-Legitimate	0.000	0.500	0.031

The ordering in Table 4 is more informative than a single winning number. Logistic Regression and Linear SVM reach similar F1 but modest PR-AUC, suggesting that the encoded transaction space is not well described by one linear boundary. The Decision Tree increases recall-oriented F1 but produces weaker ranking than the ensembles. Both Random Forest and HistGradientBoosting improve when graph context is added. The selected HGB + Graph model achieves sampled-test precision 0.670, recall 0.590, F1 0.628, and PR-AUC 0.721. These values describe a controlled, enriched modeling sample; the later stress tests determine how far that result can be trusted.

3.5 Formal Cross-validation and Grid Search

We next tune Logistic Regression, Decision Tree, Random Forest, and HGB using three-fold stratified GridSearchCV inside the 70% training partition. Imputation, scaling, and one-hot encoding are fitted within each fold. Mean validation PR-AUC is the refit criterion; after hyperparameter selection, out-of-fold training scores choose the F1 threshold, and the held-out 30% test partition is evaluated once.

Table 5: Training-only three-fold grid search and isolated test evaluation.

Model	Features	CV PR-AUC	SD	Test F1	Test PR-AUC
HGB	Retrospective graph	0.721	0.005	0.633	0.723
HGB	Base	0.561	0.009	0.545	0.556
Random Forest	Base	0.553	0.011	0.538	0.553
Decision Tree	Base	0.468	0.011	0.501	0.477
Logistic Regression	Base	0.329	0.009	0.457	0.346

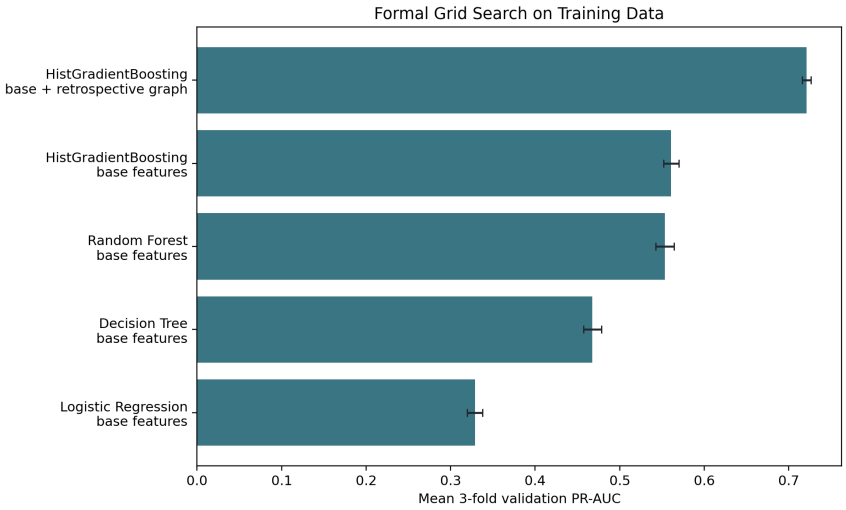
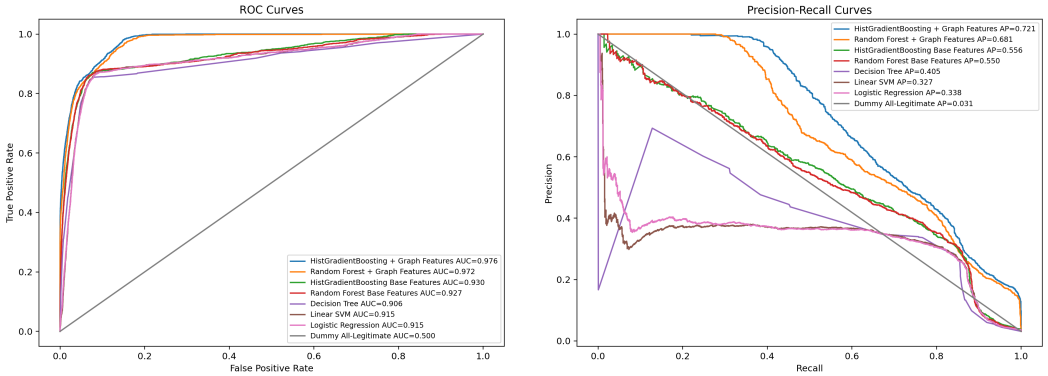


Figure 3: Mean validation PR-AUC for each best grid-search configuration; error bars are fold standard deviations. HGB with retrospective graph context is both strongest and stable within the random sampled protocol.

The best graph HGB uses learning rate 0.05, 31 leaf nodes, and L2 regularization 1.0. Its CV standard deviation of 0.005 makes the original model ordering less dependent on one validation split. Grid search nevertheless remains inside the retrospective protocol; chronology must still be tested through a different feature construction.



(a) ROC curves (b) Precision–recall curves

Figure 4: Sampled-test ROC and precision–recall curves. ROC-AUC provides the standard ranking evaluation required by the course, while PR-AUC is more informative under severe AML imbalance. Accuracy alone is misleading because an all-legitimate classifier is highly accurate on the full data but has zero recall and zero F1.

Table 6: Train, test, and full modeling-sample evaluation for the selected HistGradientBoosting + Graph Features model.

Split	Precision	Recall	F1	ROC-AUC	PR-AUC
Train	0.712	0.634	0.671	0.984	0.771
Test	0.670	0.590	0.628	0.976	0.721
Full modeling sample	0.699	0.621	0.658	0.982	0.756

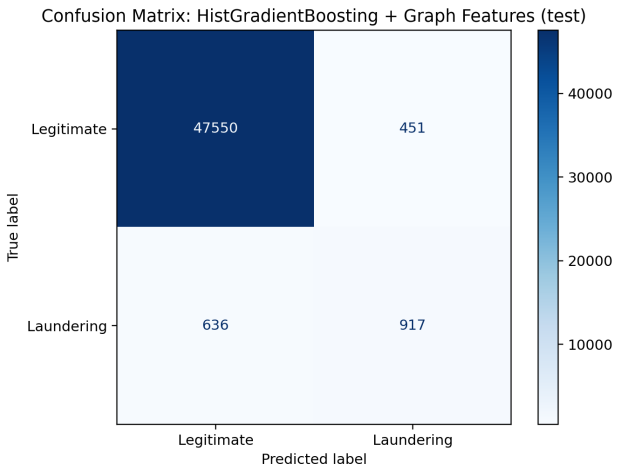


Figure 5: Sampled-test confusion matrix for the HistGradientBoosting model with graph features. This matrix makes the trade-off between recovered laundering transactions and false alerts explicit.

The sampled-test confusion matrix contains 917 true positives, 636 false negatives, 451 false positives, and 47,550 true negatives. The model therefore recovers a meaningful share of labeled laundering transactions, but it still misses 41% of positives in this sampled test and creates false alerts. Whether this balance is acceptable depends on investigation cost and regulatory risk, not only on F1. Train and full-sample confusion matrices are provided in the executed notebook. The moderate train–test gap in Table 5 also argues against describing the result as perfect fitting; the model generalizes within this protocol, but protocol shift remains the larger concern.

4 Open-ended Exploration

4.1 Account Graph Feature Engineering

Transaction-level variables answer "what happened in this payment?" Graph features ask "how does this payment fit the account's broader behavior?" We construct directed account relationships and derive in/out degree, unique counterparties, total sent/received amount, PageRank, fan-in, fan-out, and scatter-gather proxies. These features represent concentration, reach, flow asymmetry, and network position without requiring a full graph neural network.

This change in unit of analysis is the main open-ended contribution. A transfer of a moderate amount may be ordinary in isolation but unusual when an account rapidly collects from many sources and redistributes to many destinations. Conversely, a large transaction may be legitimate when it is consistent with an account's historical role. Graph context can therefore reduce ambiguity that cannot be resolved from one row. The danger is that graph statistics can accidentally summarize future or test-set edges. For that reason, the feature gain must be paired with a chronology-aware audit.

4.2 Model and Feature Ablation

The main random/transductive split result is HGB + Graph with F1=0.628, ROC-AUC=0.976, and PR-AUC=0.721. Adding graph features changes Random Forest F1 by +0.062 and PR-AUC by +0.131; for HGB the changes are +0.088 F1 and +0.166 PR-AUC. These random/transductive gains should not be overstated as deployment improvement.

The fact that two different ensemble families improve supports the hypothesis that account context contains useful signal within the random/transductive protocol. However, an ablation identifies association under a specified data construction; it does not prove causal or future predictive value. Some graph features are aggregates over relationships that span the modeling graph, so a random edge split can let training and test transactions share account context. This is why the time audit is not an optional extra check but a direct test of the mechanism behind the gain.

4.3 Mutual-information Top-k Feature Selection

Table 7: Top-k mutual-information feature selection with retraining.

Feature set	N	F1	PR-AUC	Train sec
All base+graph encoded features	60	0.628	0.721	2.41
Top-5 mutual information features	5	0.515	0.569	0.89
Top-10 mutual information features	10	0.564	0.656	0.91
Top-20 mutual information features	20	0.604	0.699	1.16
All base features (pipeline result)		0.540	0.556	
All graph features (pipeline result)		0.628	0.721	

Top-20 MI features retain most sampled-test performance while reducing feature dimensionality and training time. Their PR-AUC of 0.699 is below the all-feature value of 0.721, but training time falls from 2.41 to 1.16 seconds in this experiment. The result shows a gradual rather than binary trade-off: five features discard too much information, ten recover more, and twenty preserve most ranking quality. In a larger tuning or monitoring system, a smaller feature set could reduce maintenance and latency, but the all-feature model remains the strongest sampled-test choice here.

4.4 Natural Class-prior Stress Test

The natural class-prior stress test contains 1,501,553 rows, positive rate 0.1034%, precision 0.83%, recall 53.70%, F1 0.016, ROC-AUC 0.907, PR-AUC 0.009, FP 99,113, and TP 834. It triggers 665.6 alerts per 10,000 transactions. This experiment primarily isolates the effect of restoring the natural class prior. It is not a fully independent temporal deployment evaluation because evaluation sampling and graph-statistic construction may retain transductive elements.

This is the most important operational failure mode. A model can rank cases well on an enriched sample and still overwhelm investigators when the legitimate population returns. At the validation-selected threshold, roughly 6.7% of transactions would generate alerts, even though only about 0.1% are labeled laundering. The result does not mean the model has no value: recall remains 53.70% and ROC-AUC remains 0.907. It means that threshold choice must be linked to an explicit review budget, and that probability or score calibration under the target prior matters. Ranking quality and alert policy are separate design problems.

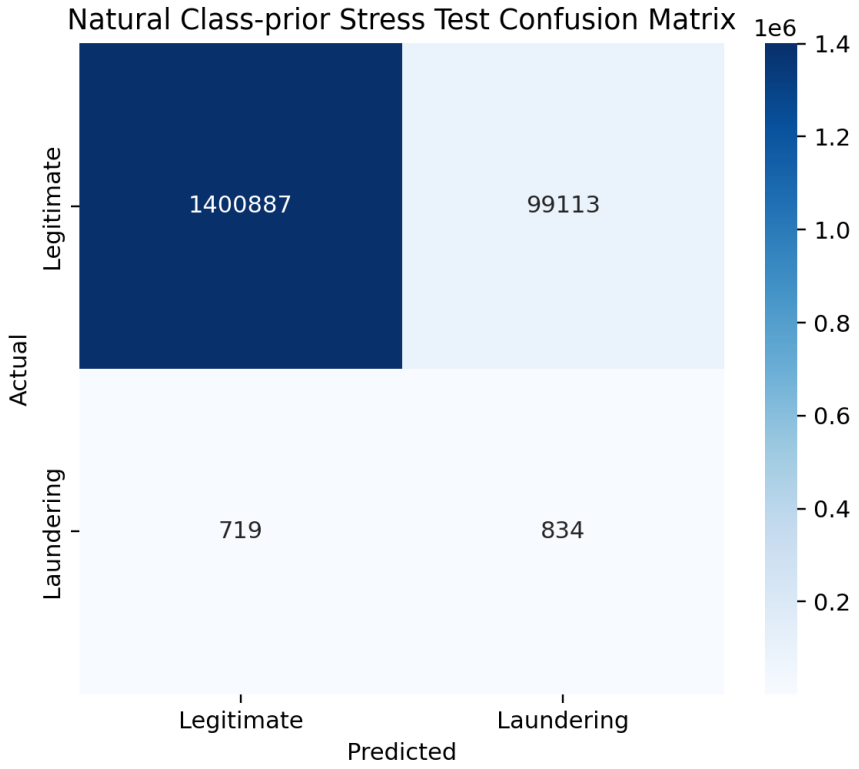


Figure 6: Natural class-prior stress-test confusion matrix. Even a strong sampled-test model produces many false positives when evaluated against the full-data base rate, which motivates alert-budget threshold analysis.

4.5 Temporal Failure, Diagnosis, and Repair

Table 8: Chronological 60/20/20 experiments: frozen-snapshot failure and strict-history repair.

Protocol	Model	F1	PR-AUC
Frozen snapshot	HGB Base	0.662	0.683
Frozen snapshot	HGB Graph	0.483	0.459
Strict expanding history	Tuned HGB Base	0.662	0.713
Strict expanding history	Tuned HGB History	0.723	0.829

The first audit maps one frozen training-period graph snapshot onto all later rows. It blocks future test edges but creates a representation mismatch: training aggregates summarize the whole training period, new accounts receive zeros, and validation/test activity never updates history. HGB Graph falls from base PR-AUC 0.683 to 0.459. We retain this failure because it reveals that "train-only" is insufficient when the representation itself should evolve online.

The repair sorts all modeling rows and records each feature before the current edge updates state. It uses prior account counts and totals, repeated and reverse edges, unique counterparties, amount deviations, recency, and 24-hour/7-day activity. Labels never enter these aggregates. Base and history models receive the same 18-candidate temporal HGB search, selected on the chronological validation period. Validation PR-AUC rises from 0.539 to 0.687; final time-test PR-AUC rises from 0.713 to 0.829 and F1 from 0.662 to 0.723.

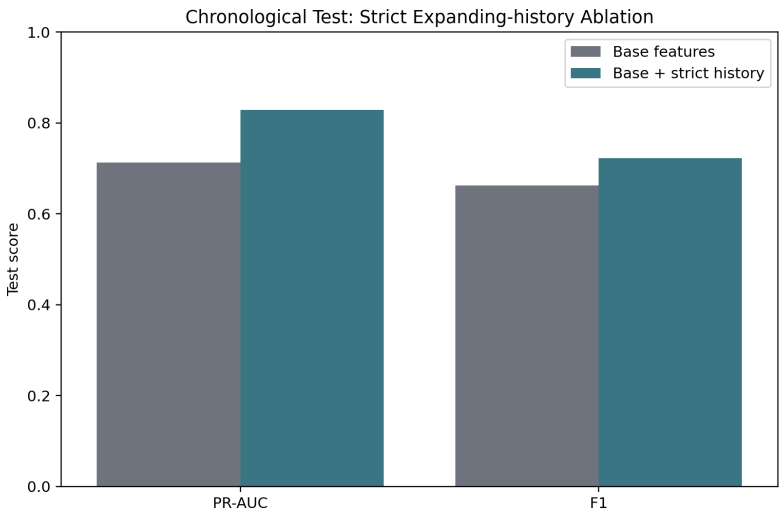


Figure 7: Strict chronological test ablation. Expanding-history features improve both PR-AUC and F1 over an independently tuned base model.

The paired PR-AUC difference is +0.116. A 500-resample paired bootstrap gives a 95% interval of [+0.100,+0.130], entirely above zero. This repairs the feature-availability failure, but it is still a modeling-sample time test with a 5.32% positive rate, not a natural-prior deployment estimate.

5 Operational and Research Implications

5.1 From Prediction to Investigation Support

The project is most useful as a study of triage design. AML analysts do not need a model that merely outputs a label; they need a ranked queue with enough contextual information to decide which cases deserve investigation. Transaction features provide event-level reasons, while graph features can provide account-level context such as unusual fan-in, fan-out, or counterparty reach. Used responsibly, such a system could help investigators prioritize network patterns that are difficult to find with fixed amount rules alone. The model should remain decision support: a flag is a prompt for review, not evidence of wrongdoing.

5.2 Why the Macro Result Matters

At a broader level, the experiments show that AML model quality depends on three interacting systems. The *statistical system* determines ranking quality; the *data system* determines whether features are available without future leakage; and the *operational system* determines how many alerts humans can investigate. Optimizing only the classifier leaves two-thirds of the problem unsolved. The natural-prior test connects scores to workload, while the time audit connects feature engineering to data availability. This systems view is more transferable than the exact score of one synthetic benchmark.

The study also demonstrates why negative findings are valuable. If we had stopped after the random split, the report would claim a graph-feature improvement without identifying its information boundary. The frozen-snapshot failure forced us to inspect the feature pipeline, distinguish a static training graph from an online history, and rerun the comparison fairly. The repaired result is stronger because it survives chronology for the right reason: every feature is available before its transaction. For organizations, the same logic argues for monitoring temporal data lineage rather than treating a one-time score as permanent evidence.

5.3 A Practical Next-stage Design

A stronger follow-up system would compute the demonstrated expanding-history features over the complete unsampled stream, add multiple rolling windows, calibrate scores on a later period with the natural prior, and select thresholds from investigation capacity. It would report performance by time period, payment format, currency, and account behavior segment to identify drift or uneven error concentration. Analysts could receive both the transaction score and a compact explanation of the relevant behavior features. Confirmed investigation outcomes would then feed a delayed, carefully governed retraining process.

6 Discussion and Limitations

The first limitation is external validity. The IBM data provide scale and complete synthetic labels, but they cannot reproduce every customer, institution, jurisdiction, or adaptive criminal strategy. A result on this benchmark should be viewed as evidence about modeling behavior, not a guaranteed real-world detection rate. Real labels are also delayed and incomplete because an alert is not the same as confirmed laundering.

The second limitation is evaluation construction. The random/transductive sample is appropriate for controlled comparison but may share account context across splits. The natural-prior stress test diagnoses base-rate effects, yet it is not a fully independent temporal test. Strict expanding history removes future-edge leakage in the chronological modeling sample, but the sampled stream omits much legitimate account history and its test positive rate is approximately 5.32%. No single protocol here simultaneously provides natural prevalence, complete prior transaction history, independent accounts, strict chronology, and realistic label delay.

The third limitation is operational modeling. We report alert pressure, but we do not have a bank-specific investigation budget, monetary cost matrix, or feedback process. Different institutions may choose very different precision–recall trade-offs. We also do not measure latency for continuously updated graph features, calibration stability, fairness across customer groups, or adversarial adaptation. These omissions bound the final claim and motivate rolling historical graph construction, out-of-time validation, account-disjoint testing, calibrated alert budgets, and human-in-the-loop evaluation.

7 Conclusion

The project completes preprocessing, t-SNE, KMeans/DBSCAN clustering, supervised training, train/test/full evaluation, ROC/AUC and PR-AUC analysis, formal three-fold grid search, graph and top-k ablations, natural-prior stress testing, temporal repair, and bootstrap uncertainty. Tuned retrospective HGB obtains mean CV PR-AUC 0.721 and test PR-AUC 0.723. More importantly, strict expanding history improves chronological test PR-AUC from 0.713 to 0.829 and F1 from 0.662 to 0.723.

The more important conclusion is methodological. Accuracy fails under extreme rarity; clustering can reveal investigation regions without matching labels; sampled ranking gains can collapse into large alert queues; and even a leakage-aware split can fail if account features are frozen rather than updated. The final positive temporal result supports graph-aware AML triage only when features are computed from strictly prior transactions. It still does not establish a deployment-ready detector: natural-prevalence, full-stream, out-of-time validation remains necessary. The project's practical value is the combination of model selection, workload awareness, failure diagnosis, and temporal data discipline—not a single headline metric.

8 References

1. IBM AML-Data GitHub repository. `https://github.com/IBM/AML-Data`
2. Kaggle IBM Transactions for Anti Money Laundering dataset. `https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml`

3. E. Altman, J. Blanuša, L. von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu. Realistic Synthetic Financial Transactions for Anti-Money Laundering Models. NeurIPS Datasets and Benchmarks, 2023.
4. B. Deprez, T. Vanderschueren, B. Baesens, T. Verdonck, and W. Verbeke. Network Analytics for Anti-money Laundering—A Systematic Literature Review and Experimental Evaluation. INFORMS Journal on Data Science, published online 2025.
5. J. Blanuša, M. Cravero Baraja, A. Anghel, L. von Niederhäusern, E. Altman, H. Pozidis, and K. Atasu. Graph Feature Preprocessor: Real-time Subgraph-based Feature Extraction for Financial Crime Detection. ACM ICAIF, 2024.
6. M. Di Gennaro, F. Panebianco, M. Pianta, S. Zanero, and M. Carminati. Amatriciana: Exploiting Temporal GNNs for Robust and Efficient Money Laundering Detection. IEEE ICDM Workshops, 2024.

9 Credit and GenAI Disclosure

Team contributions: Minghua XIONG was responsible for data processing. Bokang GAO and Zi Xuan SZETO were responsible for the remaining project components, including exploratory analysis, feature engineering, clustering, supervised modeling, model and feature ablation, natural class-prior stress testing, leakage auditing, report preparation, presentation design, and notebook integration.

GenAI disclosure: Codex/ChatGPT was used for planning, debugging, implementation assistance for model-selection and temporal-history experiments, visualization/report structuring, and language polishing. The team verified and reran the code, checked the numerical outputs, and reviewed the final interpretation. Estimated GenAI-assisted content: 25%.